Studio Nx

JS 30 — loadProjects()

2026-10-12

JS 30 — `loadProjects()`

항목	내용
파일	`studionx/frontend/static/js/pages/myproject.js`
작업	P1-14 내 프로젝트 화면: 목록 받기 (PRD 01 FR-123)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	29 `formatRelativeTime()` · 31 `renderProjects()`

1. 하는 일

검색어, 정렬, 페이지 조건으로 **내 프로젝트 목록을 서버에서 받고**, 그 조건을 기억해 **주소창에도 남깁니다**. 화면에 그리는 일은 31번 `renderProjects`가 합니다.

부른 값	서버 요청	주소창
`loadProjects()`	`/projects?query=&sort=updated&page=1&size=10`	`myProject.html`
`{ query: ' gam ', sort: 'title' }`	`query=gam&sort=title…`	`myProject.html?query=gam&sort=title`
`{ page: 2 }`	`page=2`	`myProject.html?page=2`
`{ sort: 'random', page: 'abc' }` (손으로 친 주소)	`sort=updated&page=1`로 고쳐서 보냅니다	`myProject.html`
`{ query: 'a&b%' }`	`query=a%26b%25` (안전한 글자로 바꿔 보냅니다)	`?query=a%26b%25`

실패	옮겨 가는 곳
401 (로그인이 풀림)	`home.html#login?next=/myProject.html`이며, 로그인한 뒤 돌아옵니다
500, 502 등 500번대	`errors/500.html` (503과 504는 같은 번호의 화면)
서버에 닿지 못함 (네트워크)	`errors/503.html`
그 밖의 4xx (예: 422)	`errors/400.html`

2. 언제 쓰나

호출하는 곳	화면
페이지 시작 (38)	처음 열 때 주소의 `?query=&sort=&page=`로 부릅니다
검색과 정렬 (33)	검색 칸에 입력하거나 정렬을 바꿀 때 부릅니다 (1페이지부터)
페이지 넘김 (32)	Prev, 1 2 3, Next
새 프로젝트를 만들거나 삭제한 뒤 (35, 37)	목록을 다시 받습니다. `current`의 조건을 그대로 씁니다

`history.replaceState`는 페이지를 새로 열지 않고 **주소창 글자만** 바꾸는 브라우저 기능입니다. 새로고침해도 같은 검색 결과가 나오고, 주소를 복사해서 보내도 같은 화면이 열립니다. `pushState`와 달리 뒤로 가기 기록을 쌓지 않으므로, 글자 하나를 칠 때마다 뒤로 가기 기록이 늘어나지 않습니다.

3. 코드

import (`myproject.js` 맨 위)

```
import { initAuthState } from '../core/auth-ui.js';
import { requireLogin, logout, loginUrl } from '../core/auth.js';
import { api } from '../core/api.js';
import { initModalFallback } from '../core/modal.js';
import { canOpenStudio, showDesktopOnly, initDesktopOnlyLinks, openStudio } from '../core/studio.js';
import { includePartials } from '../core/include.js';
import { initFormAlerts } from '../core/dialog.js';
```

`loginUrl`과 `api` 줄을 이번에 추가했습니다. 이 함수는 `api`(서버 요청, P1-02)와 `loginUrl`(로그인 주소 만들기, P1-02)을 씁니다.

상수 (이번에 추가)

```
/** 정렬 값 — #project-sort 의 option value · 서버 ProjectSort 와 같음 */
const SORTS = ['updated', 'title', 'created'];

/** 검색어 최대 길이 — 서버 query max_length 와 같음 */
const QUERY_MAX = 100;

/** 같은 번호의 오류 화면이 있는 상태 코드 (frontend/errors/) — userinfo.js 와 같음 */
const ERROR_PAGES = [400, 403, 404, 429, 500, 503, 504];

/** 지금 보고 있는 목록 조건 — 페이지 넘김 · 삭제 뒤 다시 받을 때 씀 (loadProjects 가 갱신) */
export const current = { query: '', sort: 'updated', page: 1 };
```

함수

```
export async function loadProjects({ query = '', sort = 'updated', page = 1 } = {}) {
  // 1. 값 정리 — 주소창에 손으로 친 값(?page=abc, ?sort=random)도 서버가 422 를 내지 않게
  query = String(query).trim().slice(0, QUERY_MAX);
  if (SORTS.indexOf(sort) < 0) sort = 'updated';
  page = parseInt(page, 10);
  if (!(page >= 1)) page = 1;   // NaN · 0 · 음수 → 1

  // 2. 서버에 요청
  const params = new URLSearchParams({ query: query, sort: sort, page: String(page), size: String(PAGE_S
IZE) });
  const ticket = ++loadTicket;
  let data;
  try {
    data = await api.get('/projects?' + params.toString());
  } catch (err) {
    if (ticket !== loadTicket) return null;   // 더 새 요청이 있으면 이 실패는 무시
    if (err.status === 401) {
      location.replace(loginUrl(location.pathname + location.search + location.hash));
    } else {
      let errorPage = err.status >= 500 ? 500 : 400;
      if (ERROR_PAGES.indexOf(err.status) >= 0) errorPage = err.status;
      else if (err.status === 0) errorPage = 503;
      location.replace('errors/' + errorPage + '.html');
```

```
    }
    return null;
  }

  if (ticket !== loadTicket) return null;   // 그사이 더 새 요청을 보냄 → 이 응답은 버림 (33)

  // 3. 지금 조건 기억 + 주소에 남김 (기본값은 빼서 주소를 짧게) — 새로고침 · 뒤로 가기에서 유지
  current.query = query;
  current.sort = sort;
  current.page = page;
  const shown = new URLSearchParams();
  if (query) shown.set('query', query);
  if (sort !== 'updated') shown.set('sort', sort);
  if (page !== 1) shown.set('page', String(page));
  const search = shown.toString();
  history.replaceState(null, '', location.pathname + (search ? '?' + search : '') + location.hash);

  return data;
}
```

최신 문법으로 쓰면 (공부용이며, 바뀌는 부분만 적었습니다)

```
const params = new URLSearchParams({ query, sort, page, size: PAGE_SIZE });
…
const shown = new URLSearchParams({
  ...(query && { query }),
  ...(sort !== 'updated' && { sort }),
  ...(page !== 1 && { page }),
});
history.replaceState(null, '', `${location.pathname}${search ? `?${search}` : ''}${location.hash}`);
if (!SORTS.includes(sort)) sort = 'updated';
```

최신 문법	실제 코드	뜻
`{ query, sort }` (줄임 속성)	`{ query: query, sort: sort }`	이름과 값이 같은 칸입니다
`...(조건 && { query })`	`if (조건) shown.set('query', query)`	조건이 맞을 때만 칸을 넣습니다
`SORTS.includes(sort)`	`SORTS.indexOf(sort) < 0`	목록에 있는지 확인합니다
템플릿 문자열 `` `${a}?${b}` ``	`a + '?' + b`	문자열을 잇습니다

문법 풀이

import 문법	뜻
`import { api } from '../core/api.js'`	`api.get(주소)`는 토큰을 붙여 보내고, 토큰이 만료되면 한 번 재발급한 뒤 다시 보냅니다. 실패하면 `ApiError`(status, code, message)를 던집니다 (P1-02)
`import { loginUrl } from '../core/auth.js'`	`loginUrl('/myProject.html')`은 `'home.html#login?next=%2FmyProject.html'`을 만듭니다 (P1-02, P1-07)

식	뜻	예
`{ query = '', sort = 'updated', page = 1 } = {}`	객체 하나를 받아서 칸을 꺼내고, 없으면 기본값을 씁니다. 아무것도 넘기지 않아도 됩니다	`loadProjects({ page: 2 })` → query는 `''`, sort는 `'updated'`

식	뜻	예
`String(query).trim().slice(0, QUERY_MAX)`	문자열로 바꾸고 앞뒤 공백을 지운 뒤 100자까지만 남깁니다	`' gam '` → `'gam'`
`parseInt(page, 10)`	문자열을 정수(10진수)로 바꿉니다	`'2'` → 2, `'abc'` → `NaN`
`!(page >= 1)`	1 이상이 **아니면** 참입니다. `NaN >= 1`은 false이므로 NaN도 걸러집니다	`NaN`, 0, −3 → 1로 바꿉니다
`new URLSearchParams({…}).toString()`	칸들을 `a=1&b=2` 모양으로 만들고, `&`, `%`, 공백은 안전한 글자로 바꿉니다	`'a&b%'` → `a%26b%25`
`err.status >= 500 ? 500 : 400`	조건 ? 맞을 때 : 아닐 때	502 → 500, 422 → 400
`location.replace(주소)`	그 주소로 옮겨 갑니다. 지금 페이지를 기록에서 **바꿔치기**하므로, 뒤로 가기로 오류 전의 페이지에 다시 오지 않습니다	
`export const current = {…}`	다른 함수(32, 35, 37)가 읽는 "지금 조건"입니다. `const`여도 안의 칸은 바꿀 수 있습니다	`current.page = 2`

4. 흐름

1. 값을 정리합니다. 검색어는 공백을 지우고 100자까지, 정렬은 3가지 중 하나, 페이지는 1 이상의 정수로 만듭니다.
2. `GET /projects?query=&sort=&page=&size=10`을 보냅니다.
3. 실패하면 401은 로그인으로, 그 밖은 오류 화면으로 옮기고 `null`을 돌려줍니다.
4. 성공하면 `current`에 조건을 기억하고, 기본값이 아닌 것만 주소에 남긴 뒤(`#…`는 그대로) 응답을 돌려줍니다.

5. 왜 이렇게

선택	이유
값을 먼저 정리	주소는 사용자가 손으로 고칠 수 있습니다 (`?page=abc`). 서버에 보내서 422를 받는 것보다 미리 기본값으로 바꾸는 편이 낫습니다. 사용자에게는 그냥 첫 페이지가 보입니다
오류는 `loadMe`(22)와 같은 규칙	템플릿에는 "500 이상 → 500"만 적혀 있었지만, 같은 모양의 페이지 데이터 API이므로 401, 503, 네트워크 오류도 같은 방식으로 처리합니다 (2026-10-06 결정: "본 데이터 API 오류 → 오류 화면")
그 밖의 4xx는 400 화면	값을 정리하므로 일어나지 않아야 하지만, 생긴다면 빈 화면보다 "잘못된 요청" 화면이 낫습니다
실패하면 `null`을 돌려줌	부르는 쪽(38 등)이 `if (!data) return;` 한 줄로 멈출 수 있습니다. 오류 화면으로 옮겨 가는 중에 그리기를 하지 않게 합니다
주소에는 **기본값을 뺀** 것만 남김	`myProject.html?query=&sort=updated&page=1`보다 `myProject.html`이 깔끔합니다. 읽을 때(38)는 값이 없으면 기본값을 씁니다
`replaceState`를 씀 (`pushState`가 아님)	검색할 때마다 뒤로 가기 기록이 쌓이지 않게 합니다
주소 갱신은 **성공한 뒤에**	실패해서 오류 화면으로 가는데 주소만 바뀌는 일이 없게 합니다
`current`를 밖으로 내보냄	페이지 넘김, 새 프로젝트, 삭제 뒤에 "같은 조건으로 다시 받기"를 할 때 씁니다

선택	이유
옛 응답 버리기 (`loadTicket`, 33에서 추가)	빠르게 검색하면 먼저 보낸 요청이 나중에 도착해서 새 결과를 덮어쓸 수 있습니다. 그래서 요청마다 번호를 붙이고 마지막 것만 씁니다 (33 §5)
`ERROR_PAGES`를 userinfo.js와 따로 둠	지금은 두 곳뿐이라 그대로 두었습니다. 세 번째 페이지(P1-17 myPage)에서도 쓰게 되면 `core/`로 옮기는 것이 좋습니다

6. 시험 (실제 myProject.html, 실제 서버, 헤드리스 Chrome, 14개)

시험 계정 `sitetest.p114@example.com`으로 가입하고 로그인한 뒤 프로젝트 3개(Beta → alpha → Gamma 순서로 만듦)를 만들고, 페이지 안에서 함수를 불렀습니다. 오류 시험은 `window.fetch`를 가짜로 바꿔서 서버 응답을 흉내 냈습니다. 끝난 뒤 계정과 프로젝트를 지웠습니다.

#	부른 값	결과와 주소
01	기본	`Gamma, alpha, Beta`(최근 수정순), total 3, 주소 `myProject.html`
02	`query: ' gam ', sort: 'title'`	`Gamma`, current `{query:'gam', sort:'title', page:1}`, `?query=gam&sort=title`
03	`sort: 'random', page: 'abc'`	updated와 1로 고쳐서 3개를 받고, 주소는 깨끗합니다
04	`page: 0`, `page: -3`	둘 다 1이 됩니다
05	`page: 2` (10개씩이라 없는 쪽)	빈 목록, total 3, `?page=2`
06	`query: 'a&b%'`	0개, `?query=a%26b%25` (특수 문자가 주소를 깨지 않습니다)
07	150자 검색어	100자로 잘라 보내서 422 없이 0개를 받습니다
08	주소 `?query=old&page=9#top`에서 `sort: 'created'`	`?sort=created#top`이 됩니다. 옛 값은 지우고 `#top`은 남깁니다
09	가짜 500	`errors/500.html`
10	가짜 503	`errors/503.html`
11	가짜 502	`errors/500.html`
12	가짜 422	`errors/400.html`
13	네트워크 끊김 (fetch 실패)	`errors/503.html`
14	로그아웃한 뒤	`home.html#login?next=%2FmyProject.html`